# Agent Development Methodology - I

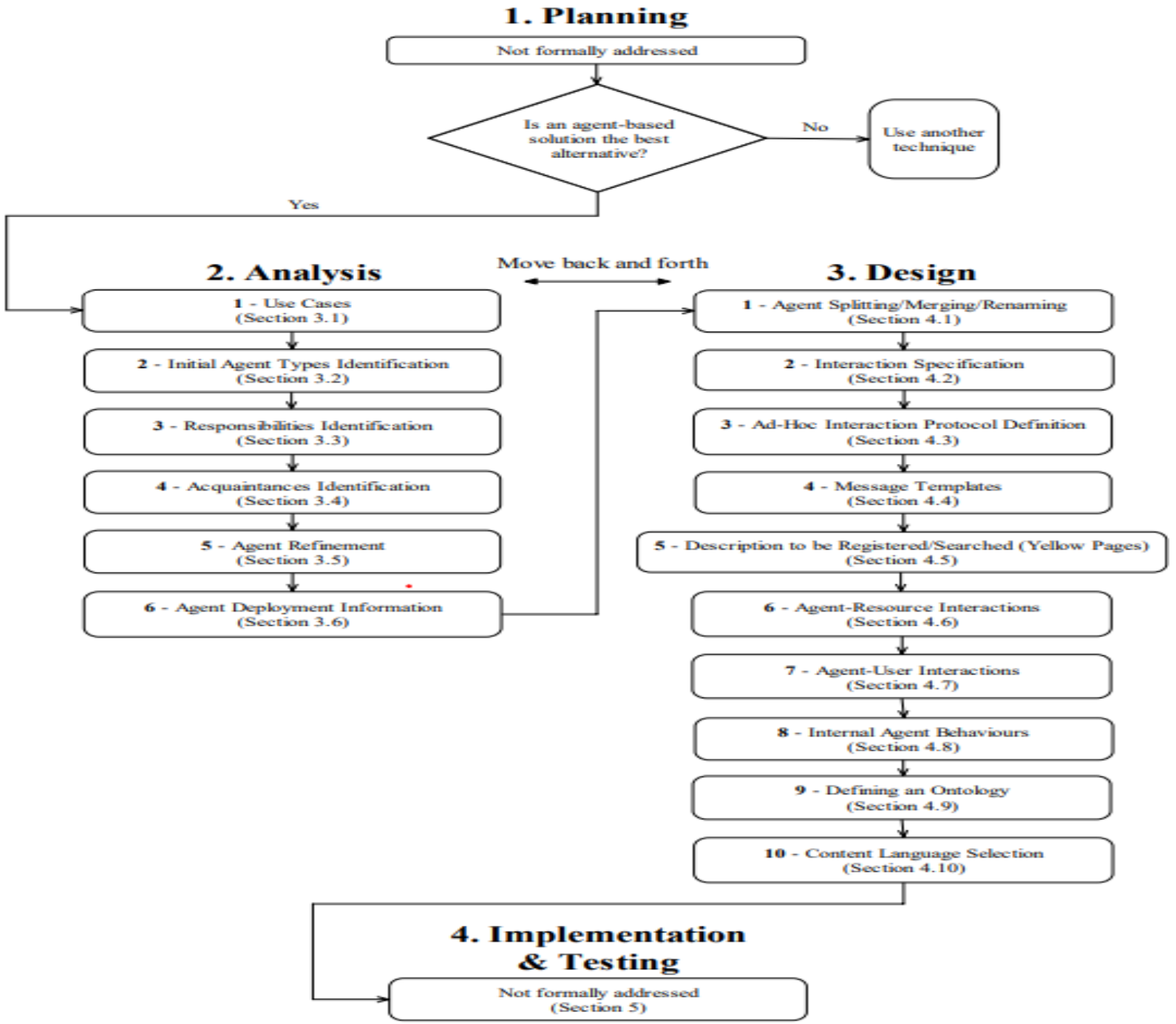
Dr. Kaneeka Vidanage

# What are Agents & Agent Attributes ?

- Software agents are application programs that communicate with each other in an expressive agent communication language.
- Agents reside on a platform that, consistent with the presented vision, provides the agents with a proper mechanism to communicate by names, regardless of the complexity and nature of the underlying environment (i.e., operating systems, networks, etc.)

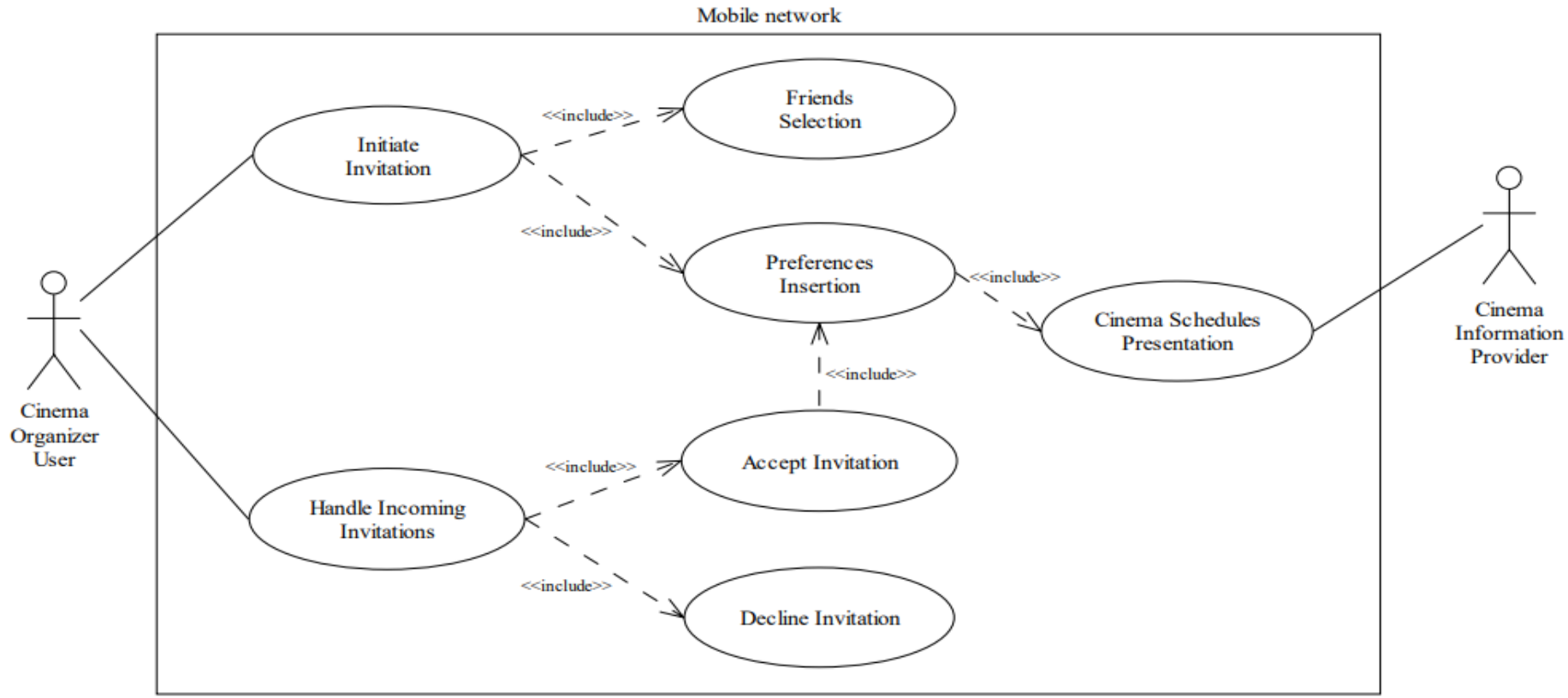
  - *Autonomy*: agents operate without the direct intervention of humans or others, and have some kind of control over their actions and internal state.
  - *Social ability*: agents interact with other agents (and possibly humans) via some kind of agent communication language.
  - *Reactivity*: agents perceive their environment and respond in a timely fashion to changes occurring therein.
  - *Pro-activeness*: in addition to acting in response to their environment, agents are able to exhibit goal-directed behaviour by taking the initiative.

# Agent Development Methodology



# Cinema Organizer Case Study

- This case study will support a group of friends in organizing an evening to the cinema. The service should allow a subscriber to:
  - invite some friends to see a movie
  - collect preferences from both the inviter and the invitees, and
  - suggest the option that best matches the average preferences of the group

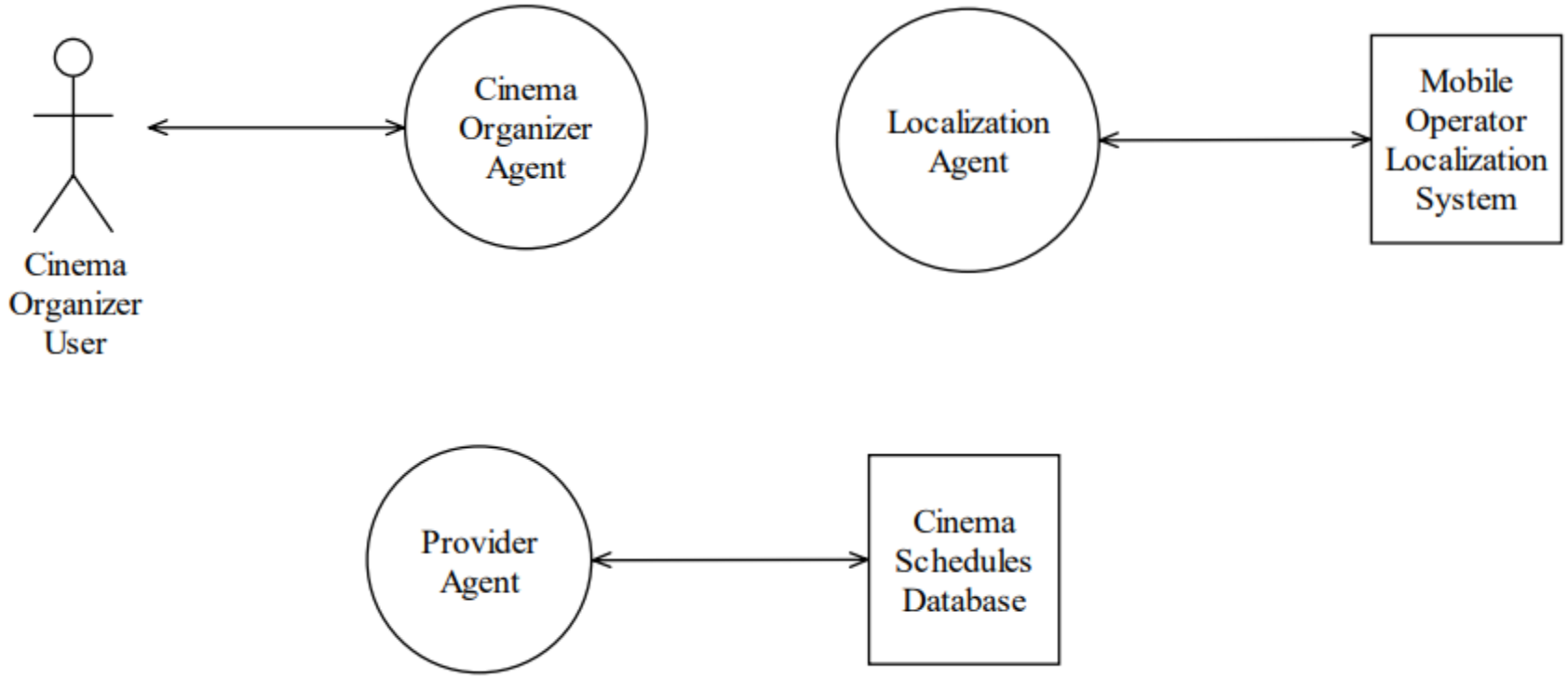


# Agent Diagram

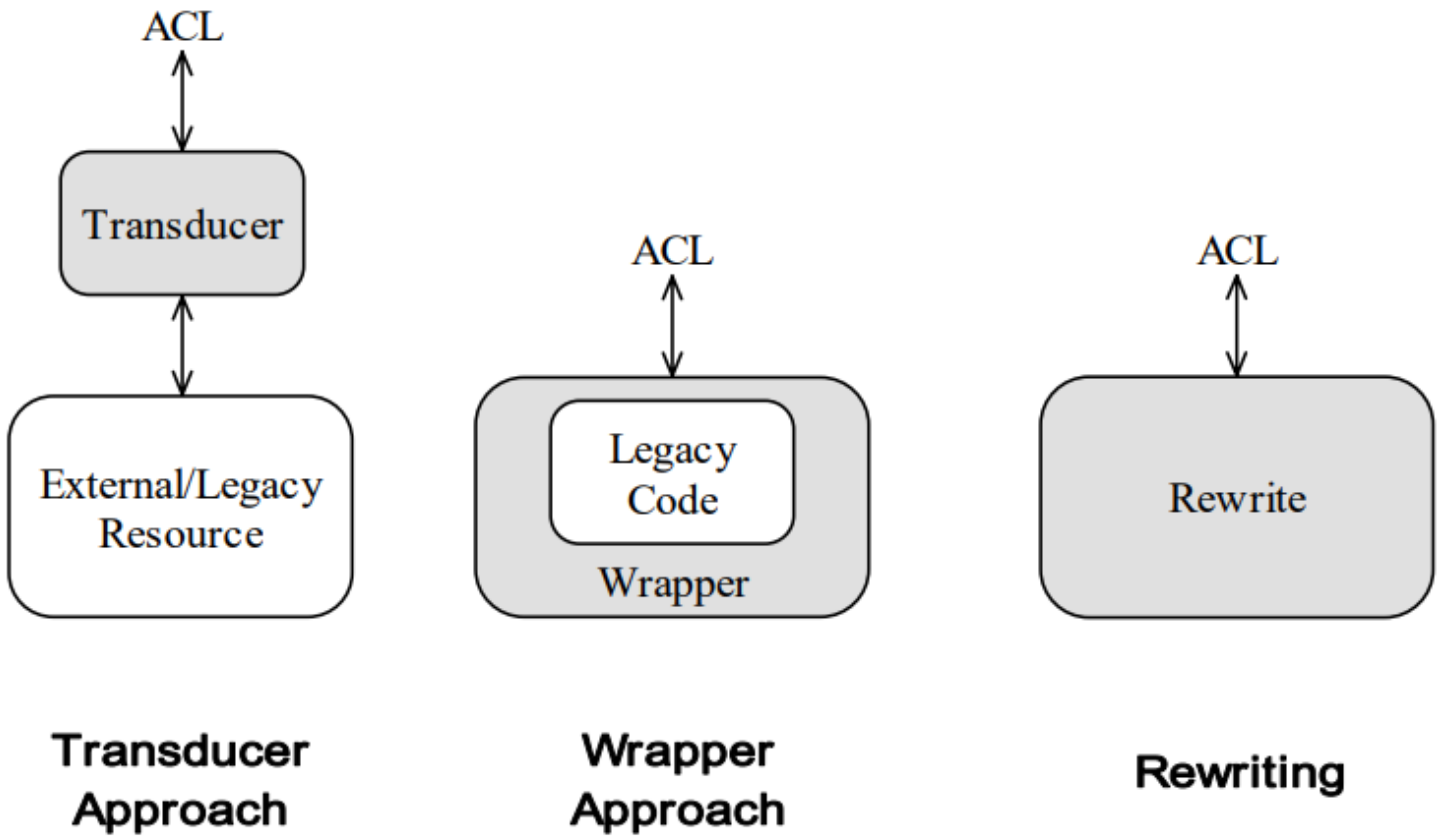
1. *Agent types*: the actual agent types, represented by circles.
2. *Humans*: people that must interact with the system under development, represented by the UML actor symbol.
3. *Resources:* external systems that must interact with the system under development, represented by rectangles.
4. *Acquaintances:* represented by an arrow linking instances of the above elements, specifying that the linked elements will have to interact in some way while the system is in operation. Note that, at this stage, only acquaintances between agents and resources/humans are shown in the agent diagram (i.e. agent-agent interactions are deferred to a later step).

# Initial Agent Types Identification

- Add one type of agent per user/device.
- Add one type of agent per resource



# Options for external / legacy systems to get intact with Agents

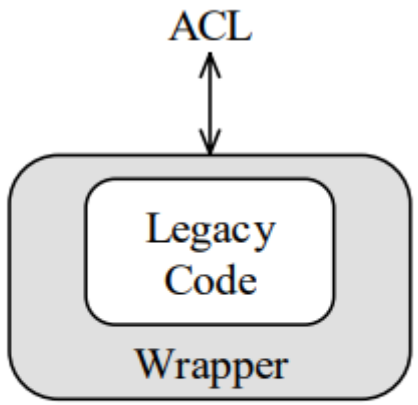


# Transducer Approach

The use of a *transducer agent*. The transducer agent serves an interface between a legacy system and the other agents in the system. The transducer agent accepts messages from the agents in the system (in agent communication language), translates them into the legacy systems native language, and forwards these equivalent messages to the legacy system. Similarly, in the reverse direction, the transducer agent receives the legacy systems responses and makes them available to the other agents in the system. In addition to acting as an interface between agents and legacy software, the transducer approach also works for other resources such as files and people

# Wrapper Approach

The insertion of a *wrapper*. A code is injected into the legacy resource (i.e. software in this case), provided the legacy resource's code is available. This inserted code will allow the resource to communicate in agent communication language, thus, converting it into an agent.




**Wrapper Approach**

# Rewriting Approach

*Rewriting* of the code. This is the most extreme approach, which involves rewriting the code to mimic (and possibly extend) the operation and capabilities of the legacy resource (i.e. software in this case), but with the added ability to communicate in agent communication language. Note that this approach is usually the last resort, when no other options (i.e. the use of a transducer or a wrapper) are deemed practical.

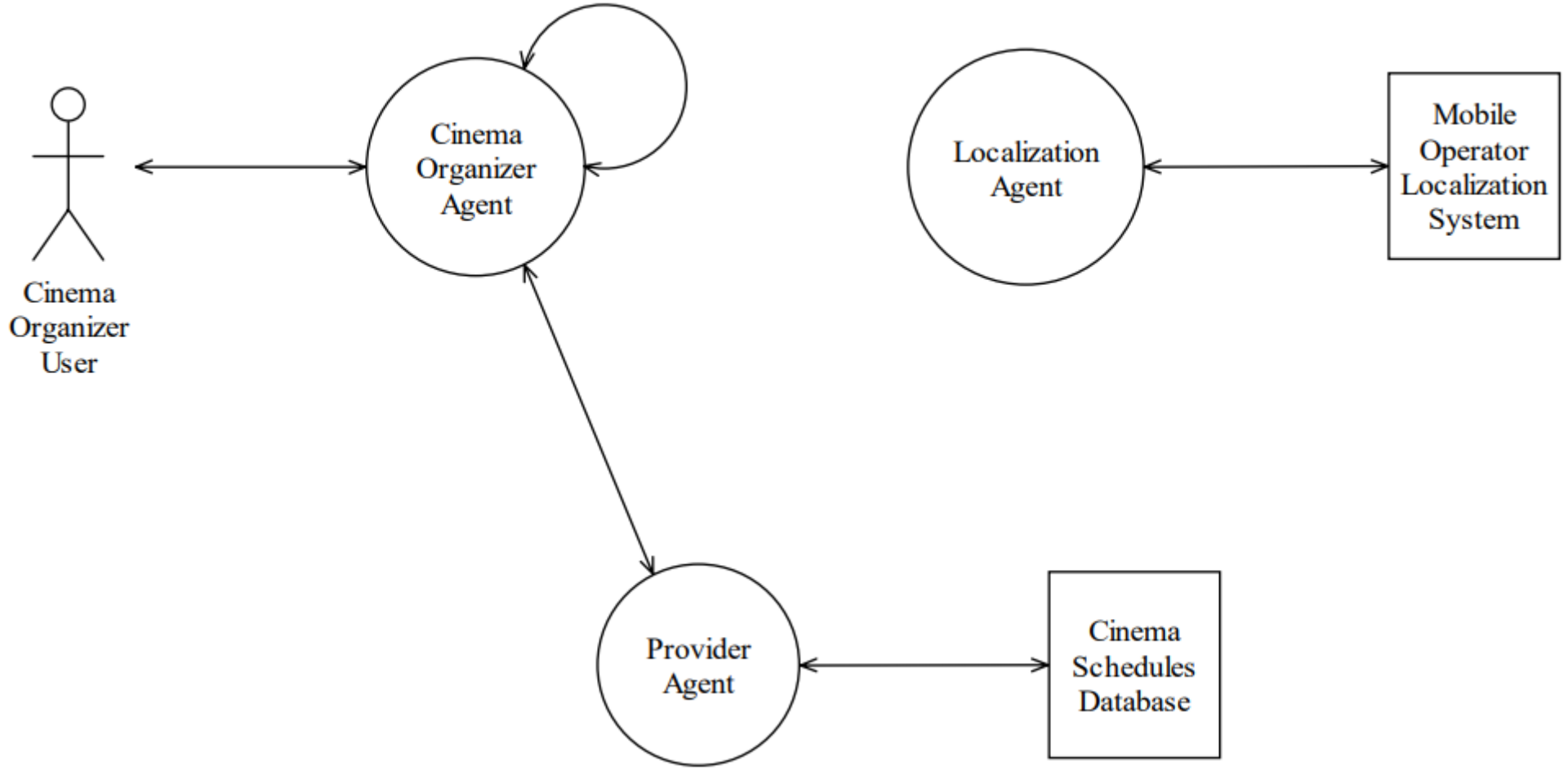
# Responsibilities Identification

- In this step, for each identified agent type, an initial list is made of its main responsibilities in an informal and intuitive way. The artifact resulting from this process is the responsibility table.

| **Agent type** | **Responsibilities** |
|---|---|
| Cinema organizer agent | Serve requests to initiate invitations from the cinema organizer user.<br>Let the cinema organizer user select friends to invite.<br>Let the user insert preferences about movies and cinemas.<br>Present cinema schedules.<br>Present incoming invitations from other cinema organizer agents.<br>Let the user accept an incoming invitation.<br>Let the user reject an incoming invitation.<br>Retrieve cinema schedules from the relevant provider agent. |
| Provider agent | Respond to cinema schedules retrieval requests from cinema organizer agents. |

# Acquaintances Identification

- Focus is on who needs to interact with whom



# Agent Refinement

- In this step, the set of agent types initially identified in Step 2 are refined by applying several considerations. These are related to :-
  - Support: what supporting information agents need to accomplish their responsibilities, and how, when and where is this information generated/stored.
  - Discovery: how agents linked by an acquaintance relation discover each other
  - Management and monitoring: is the system required to keep track of existing agents, or the starting and stopping of agents on demand

# Support

These considerations are highly dependent on the domain, and hence, it is quite difficult to provide generic indications. In the cinema organizer case study, the only additional information with respect to what has already been taken into account is that the cinema organizer agent requires the city it is currently in. Thus, the cinema organizer agent is required to question the provider agent for details of the current city when retrieving the cinema schedules information. This information, however, can be obtained by accessing the localization system through the localization agent and therefore there is no need for any new agent type in this case.

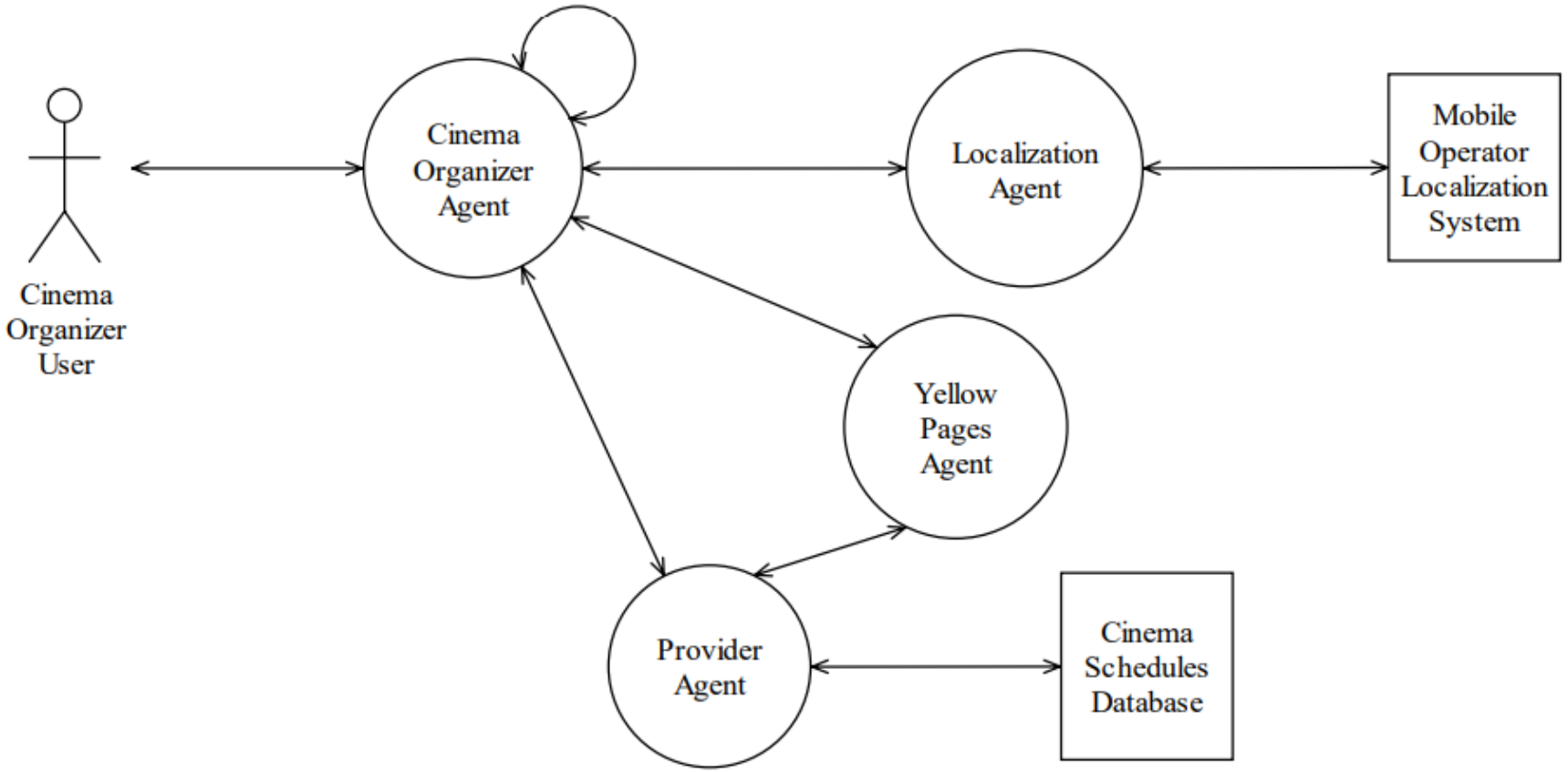
# Discovery

A more sophisticated way to solve the agent discovery problem is the adoption of a *yellow pages mechanism*. This allows discovery of agents on the basis of their characteristics, e.g. the services they provide. A yellow pages mechanism can be fully distributed across all agents in the system or centralized with a single agent (with a well-known name) responsible for it. Even if this choice, at this point, is a high level design choice, considering that the proposed methodology targets the JADE platform, it is strongly suggested to adopt a centralized approach. This approach completely maps to the directory facilitator agent provided by JADE and thus saves a lot of work in successive phases of the development process.

# Management and Monitoring

Other agent types can be added to address issues such as monitoring agent faults and restoring them, creation of supporting agents that are needed only under certain conditions, or providing presence information. No new agent types need to be added in the cinema organizer case study for management and monitoring purposes.

# Modified Agent Diagram with new additions

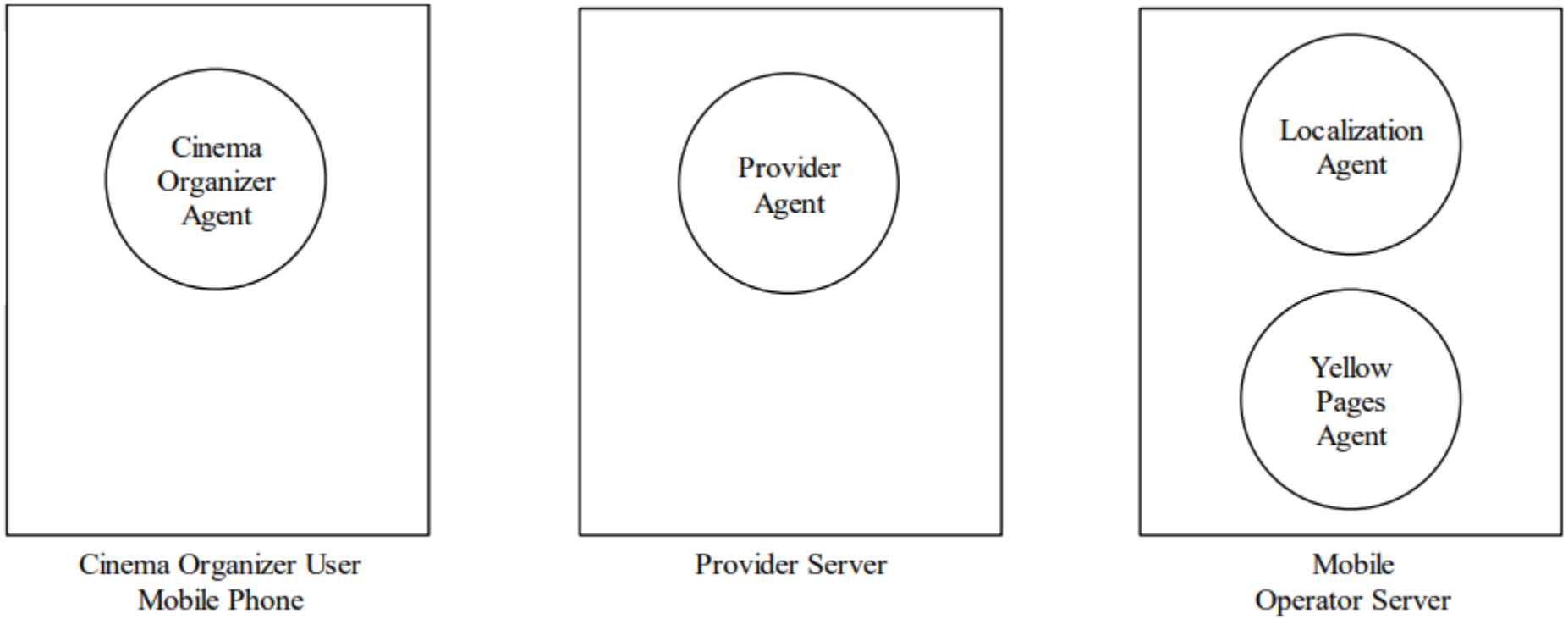


# Modified Responsibility Table

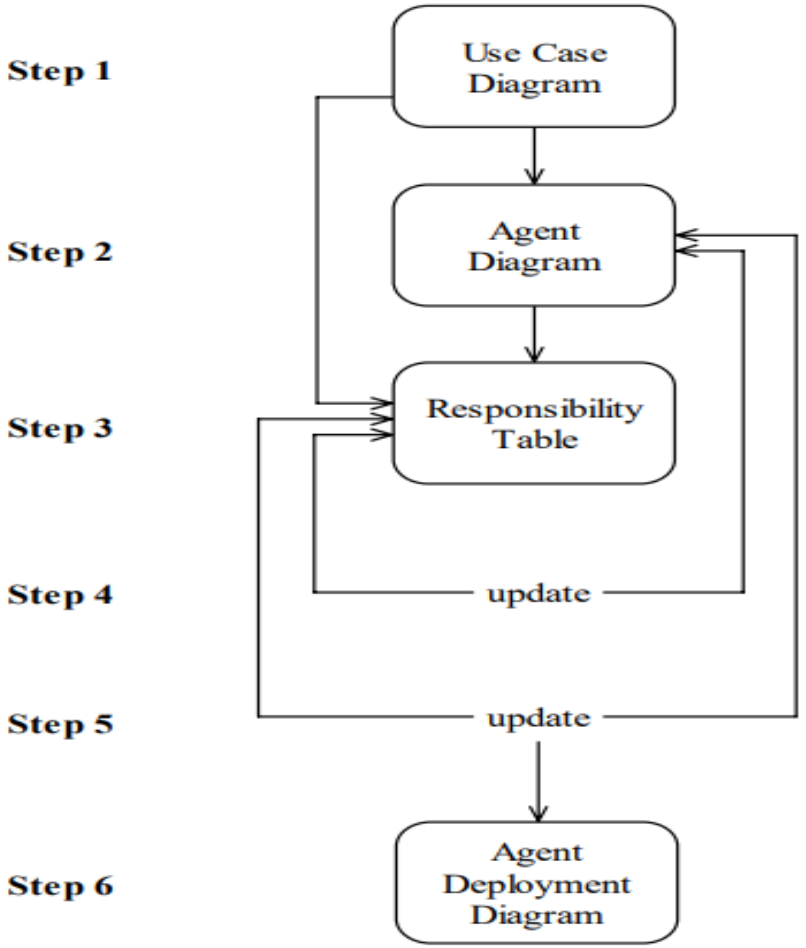
| Agent type | Responsibilities |
|---|---|
| Cinema organizer agent | 1. Serve requests to initiate invitations from the cinema organizer user. |
| | 2. Let the cinema organizer user select friends to invite. |
| | 3. Let the user insert preferences about movies and cinemas. |
| | 4. Present cinema schedules. |
| | 5. Present incoming invitations from other cinema organizer agents. |
| | 6. Let the user accept an incoming invitation. |
| | 7. Let the user reject an incoming invitation. |
| | 8. Retrieve cinema schedules from the relevant provider agent. |
| | 9. Retrieve the current city from the localization agent. |
| | 10. Retrieve the relevant provider agent from the yellow pages agent. |
| Provider agent | 1. Respond to cinema schedules retrieval requests from cinema organizer agents. |
| | 2. Register with the yellow pages agent. |
| Localization agent | 1. Respond to city retrieval requests from cinema organizer agents. |
| | 2. Notify city changed to cinema organizer agents. |

# Agent Deployment

- Another artifact that can be useful to produce is the agent deployment diagram, where the physical hosts/devices agents are going to be deployed (referred to as domains in some methodologies) are indicated.



# Summary of the Analysis Phase



# Design Phase

- This step is considered important, since it has a direct effect on overall system efficiency and complexity.
- Following rules should be applied in this step :-
  - Data duplication should be avoided. If there are two or more agents that share a large majority of the information required to carry out their tasks, these agents can possibly be merged into a single one.
  - Duplication of code to access resources should be avoided. If there are two or more agents that need to access the same resource, these agents can possibly be merged.
  - Avoid splitting agents unless there are good reasons for doing this. Dealing with too many agents increases the overall system complexity and decreases system efficiency since unnecessary communication between agents will possibly take place.
  - Each agent is situated on a single machine. If two pieces of functionality must be provided on different machines, these pieces of functionality must be provided by different agents.
  - Avoid having agents that are too big and complex. This makes them difficult to design and to maintain

# Interactions Specification

- An interaction table is produced for each agent type. Each row in the table will represent an interaction and will include :-
  - A descriptive name for the interaction
  - The responsibility that originates this interaction → Linked with the responsibility table created above.
  - A suitable interaction protocol (IP) chosen to implement the interaction. The standard FIPA interaction protocols should be considered as best candidate.
  - The role played by the considered agent in the interaction protocol. This can be I for initiator or R for responder. Other roles can possibly be added if required.
  - The agent type and name (if relevant) of the complementary role.
  - The trigger condition, i.e., when this interaction takes place. This condition should be expressed in an informal but descriptive way

# Interaction Table

| Interaction | Resp. | IP | Role | With | When |
|---|---|---|---|---|---|
| Invite other users. | 1 | Contract Net | I | Cinema organizer agent | The user initiates an invitation. |
| Respond to an invitation. | 5 | Contract Net | R | Cinema organizer agent | An invitation is received. |
| Retrieve cinema schedules. | 8 | FIPA Request | I | Provider agent of the current city | - Startup.<br>- A city change is detected and the provider for the new city has been retrieved. |
| Retrieve the current city. | 9 | FIPA Subscribe | I | Localization agent | Always. |
| Retrieve the provider agent for a given city. | 10 | FIPA Request | I | Yellow pages agent | - Startup.<br>- A city change is detected. |

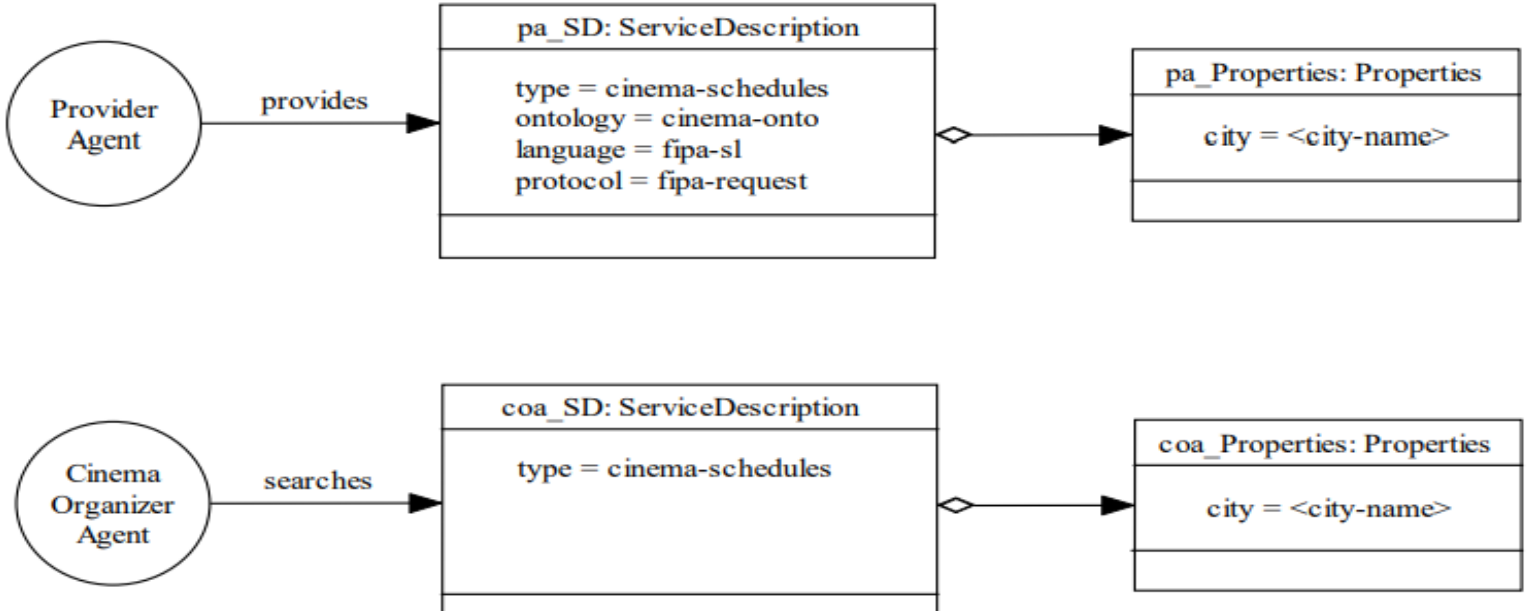
# Message Templates

- Use Message Templates based on the conversation ID in behaviors implementing initiator roles.
- Merge in a single behavior responder roles dealing with the same combination of initiation message performative, ontology, and language.
- Analyze conflicts and modify Message Templates as needed.

| Interaction | Resp. | IP | Role | With | When | Template |
|---|---|---|---|---|---|---|
| Invite other cinema organizer agent users. | 1 | Contract Net | I | Cinema organizer agent | The user initiates an invitation. | Conv-id |
| Respond to an invitation. | 5 | Contract Net | R | Cinema organizer agent | An invitation is received. | Perf = CFP |
| …… | | | | | | |

# Yellow Pages

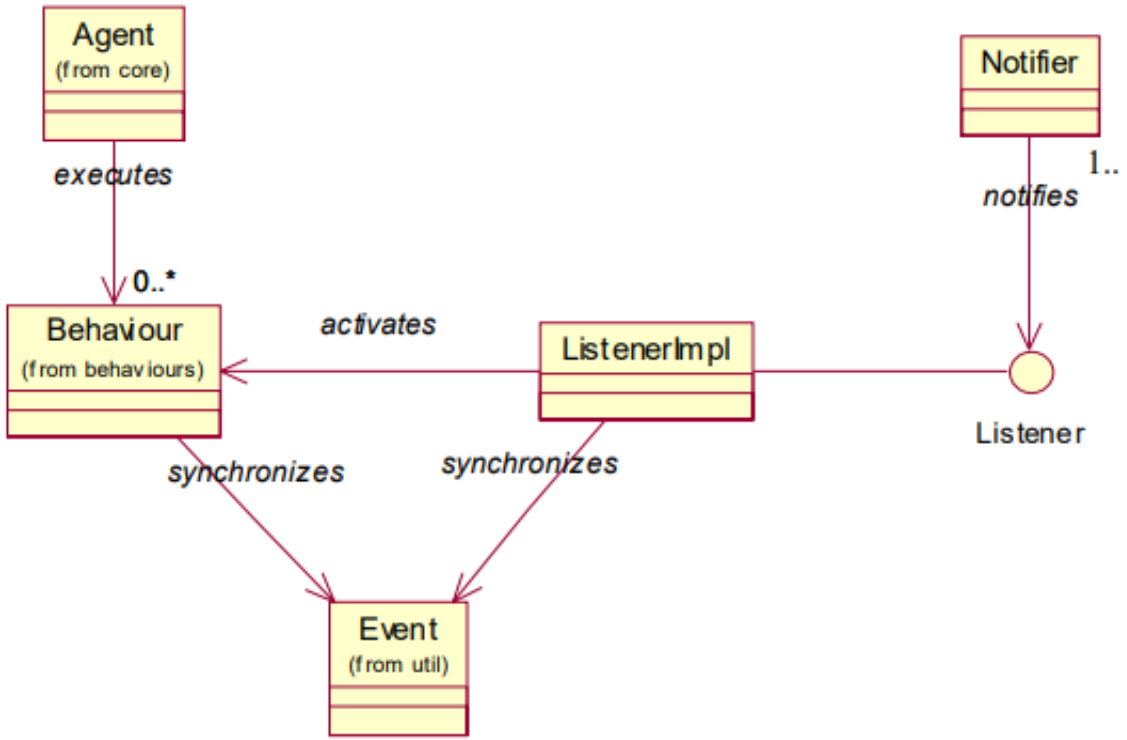
- In this step, the naming conventions and the services registered/searched by agents in the yellow pages catalogue maintained by the JADE directory facilitator are formalized (when relevant).
- Naming conventions are mainly domain dependent, and it is suggested to use natural language to specify them. Furthermore, a class diagram form is proposed to describe service registrations/searches



# Agent- Resource Interactions

- It is often the case that one or more agents in the system must interact with external resources such as databases, files storing information, or legacy software. In some cases, some hardware appliances must be controlled or monitored, but this always happens through some dedicated software that actually hides the hardware behind it.
- Such resources can be classified into two main categories :-
  - Passive resources: resources that change their status only as a consequence of some stimulus issued by the agent controlling the resource itself. → Motion detection CCTV control
  - Active resources: resources that may change their status independently from the controlling agent. → CCTV ration is always controlled despite a motion.

# Agent – Passive Resource Connectivity



# Querying a Relational Database

- The approach proposed is to stick to the transducer approach and avoid embedding JDBC code inside all agents that may need to query the database.
- The proposed solution preserves the full power of SQL/JDBC without the need for embedding JDBC code (and the related complexity) inside all agents that need to perform queries on the database.

